How ASIM normalizes data in Microsoft Sentinel

The schemas are generic. The parsers are written for each product and each table.

SOURCE TABLES

- `DeviceProcessEvents`
- `SecurityEvent 4688`
- `Sysmon event ID 1`
- `CrowdStrike Falcon`

Each product writes its own columns.

SOURCE PARSERS

- Defender XDR parser
- Windows 4688 parser
- Sysmon parser
- CrowdStrike parser

One per product, per schema. Reads exact column names.

UNIFYING PARSER

`_Im_ProcessCreate`

Takes filter parameters and passes them down to each source parser:

- `starttime, endtime`
- `commandline_has_any`

Unions every source parser for the schema.

CONTENT

- Analytics rules
- Hunting queries
- Workbooks

Written once against the unifying parser.

Depends on the product and the table schema

Same for every product

WHAT ONE SOURCE PARSER DOES: DEFENDER XDR TO THE PROCESS EVENT SCHEMA

DeviceProcessEvents column		ASIM field
`InitiatingProcessFileName`	→	`ActingProcessName`
`ProcessCommandLine`	→	`TargetProcessCommandLine`
`AccountName`	→	`ActorUsername`
`DeviceName`	→	`DvcHostname`
`SHA256`	→	`TargetProcessSHA256`

It also sets `EventType` (for example ProcessCreated), `EventVendor`, `EventProduct` and `EventSchemaVersion`.

No parser for a source

ASIM content returns nothing for that source.

Source schema changes

The parser usually returns no rows, with no error.

Ingest-time option

A DCR transform writes native ASim tables such as `ASimProcessEventLogs`.

Sources: Microsoft Learn, ASIM schemas and ASIM parsers overview

Checked 16 October 2026

© 2026 LevelBlue Intellectual Property. LevelBlue logo, and registered trademarks and service marks are the property of LevelBlue Intellectual Property and/or LevelBlue affiliated companies. All other marks are the property of their respective owners.

ASIM parsers do not run on the Sentinel data lake

Lake KQL does not support functions, and every ASIM parser is a function.

ANALYTICS TIER

- Hunting query or analytics rule
- Calls `_Im_NetworkSession()`
- Source parsers run (KQL functions)
- Tables return ASIM-shaped rows

DATA LAKE TIER

- Hunting query or KQL job
- Calls `_Im_NetworkSession()`
- Source parsers never run
- No ASIM-shaped rows

> “Using out-of-the-box or custom functions isn't supported in KQL queries against the data lake.”
>
> Microsoft Learn, KQL queries in the Microsoft Sentinel data lake. Every ASIM parser is a function.

ALSO NOT SUPPORTED IN LAKE KQL

`externaldata()` `adx()` `arg()` `ingestion_time()` `estimate_data_size()`

AzureDiagnostics and other legacy tables

WHAT STILL RUNS ON THE LAKE

- KQL against the raw tables
- Native ASim tables, already in ASIM shape
- KQL jobs with parser logic pasted in as plain KQL
- Spark notebooks

15 min
before ingested data can be queried

500,000 rows
or 64 MB per interactive result

30 / min, 10 at once
query limits per tenant

SAME HUNT, REWRITTEN FOR THE LAKE

The parser's mapping travels inside the query as plain KQL.

```
CommonSecurityLog
| where DeviceVendor == "Palo Alto Networks"
| project-rename SrcIpAddr = SourceIP,
    DstIpAddr = DestinationIP, DvcHostname = DeviceName
| extend EventVendor = "Palo Alto Networks"
```

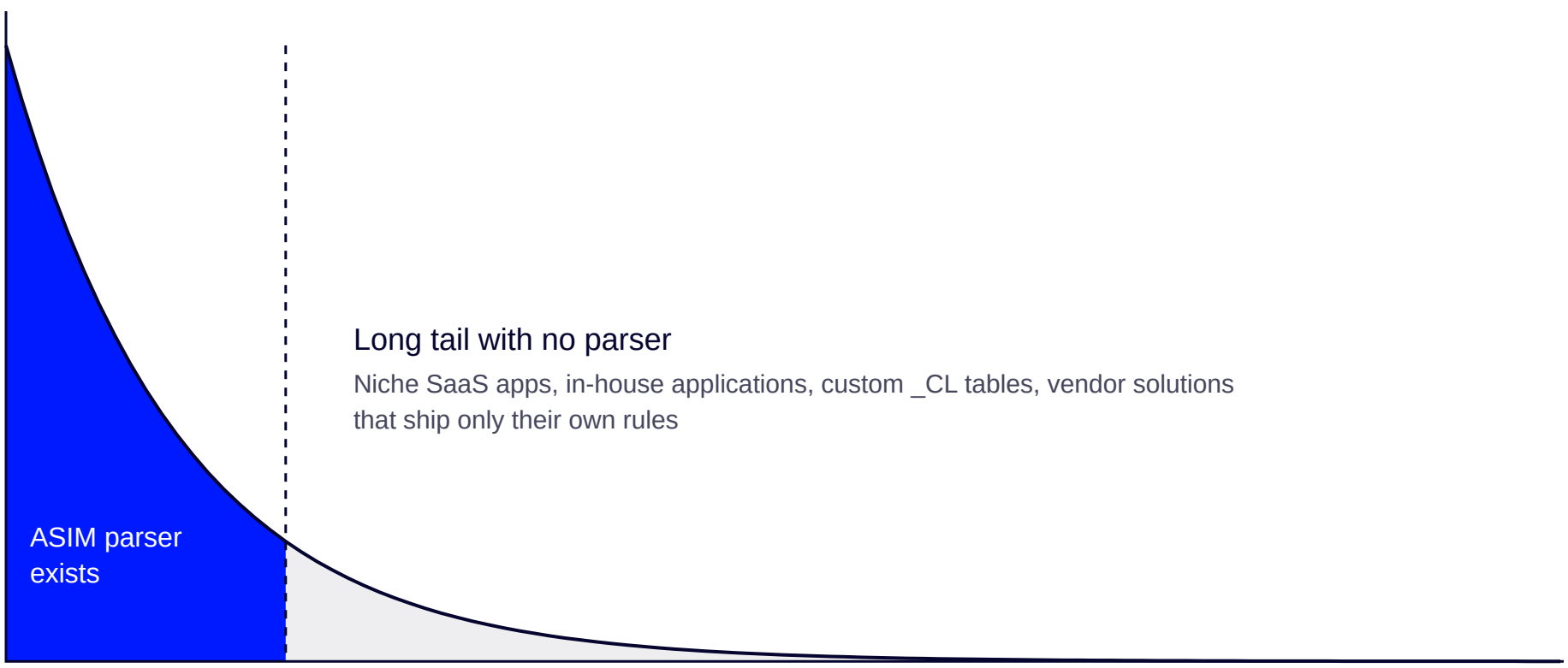
Sources: Microsoft Learn, KQL queries in the data lake; Troubleshoot KQL queries for the data lake

Checked 16 October 2026

© 2026 LevelBlue Intellectual Property. LevelBlue logo, and registered trademarks and service marks are the property of LevelBlue Intellectual Property and/or LevelBlue affiliated companies. All other marks are the property of their respective owners.

ASIM coverage depends on which parsers exist

The schemas are product-neutral, but a source only gets ASIM content once someone writes its parser.



A parser exists

Every ASIM analytics rule, hunting query and workbook covers the source with no extra work.

No parser exists

ASIM content returns nothing for the source. Someone has to write and maintain a parser: Microsoft, the vendor, the community or you.

The vendor solution ships only its own rules

Those rules query the raw table directly. They do not feed or use ASIM content.

USUALLY COVERED BY ASIM PARSERS

- Major firewalls
- Web proxies
- DNS servers
- EDR platforms
- Linux sshd and su/sudo
- Cloud audit logs

USUALLY NOT COVERED

- Niche SaaS apps
- In-house applications
- Custom _CL tables
- OT and IoT gateways
- Vendor solutions without parsers

Sources: Microsoft Learn, ASIM parsers overview; Microsoft Tech Community, ASIM additions (June 2026)

Checked 16 October 2026

© 2026 LevelBlue Intellectual Property. LevelBlue logo, and registered trademarks and service marks are the property of LevelBlue Intellectual Property and/or LevelBlue affiliated companies. All other marks are the property of their respective owners.

Six ways to get common fields, compared at scale

What each method gives you, what it costs to maintain, and whether it runs on the lake.

● ◑ ○ Fields: full to none ● ◔ ○ Upkeep: heavy to none

METHOD	FIELDS YOU GET	UPKEEP AT HUNDREDS OF SOURCES	RUNS ON THE LAKE	USE IT FOR
Per-source DCR into ASim or custom tables Ingest time	● Full ASIM fields	● One DCR per source	✓ Yes, check table tier support	Top 10 to 20 sources by volume
ASIM parsers at query time Query time	● Full ASIM fields	◔ Microsoft maintains	⊗ No	Analytics-tier tables only
ASIM parser logic inlined in KQL jobs or notebooks Query time	● Full ASIM, where a parser exists	◔ Scheduled sync from the Azure-Sentinel GitHub repo	✓ Yes	Sources Microsoft already parses
Discovery scan and entity index Scheduled job	◑ IP, user, host, hash, URL	◔ One synonym dictionary	✓ Yes	The long tail of tables
Syslog template mining Scheduled job	◕ Fields per message template	◑ Review new templates	✓ Yes	Unix syslog, auditd, raw text
Free-text search Query time	◔ Literal indicator matches	○ None	✓ Yes, slow; test union * first	Backstop for indicators

ROUTE EACH SOURCE TO A METHOD

Source has an ASIM parser	Inline the ASIM parser logic	Tier 1
Structured fields, no parser	Discovery scan and entity index	Tier 2
Raw text: syslog, auditd	Template mining, then value regex	Tier 2
Any source, any time	Free-text search as a backstop	Tier 3

Mix for hundreds of sources: inlined ASIM logic, the entity index, template mining and free-text search.

Keep per-source DCRs for the few highest-volume sources. Writing and maintaining hundreds of them does not scale.

Sources: Microsoft Learn, ASIM and data lake KQL documentation; LevelBlue analysis

Checked 16 October 2026

© 2026 LevelBlue Intellectual Property. LevelBlue logo, and registered trademarks and service marks are the property of LevelBlue Intellectual Property and/or LevelBlue affiliated companies. All other marks are the property of their respective owners.

Find the common fields, then index them

Read three kinds of field, classify the values, and write one entity index for hunting.

Every table in the lake

Microsoft tables, CommonSecurityLog, Syslog, custom _CL tables. A Spark notebook lists and samples them without the interactive row cap.

Typed columns

Read the table schema (`getschema`, `df.schema`)

SigninLogs.UserPrincipalName

JSON and dynamic columns

Pull keys from sampled rows (`bag_keys`)

AdditionalFields, RawEventData, Properties

Key=value and raw text

`parse_kv`, regex, template mining

AdditionalExtensions, SyslogMessage, RawData

Field inventory

One row per table and field path, with type, fill rate and a sample value. Example rows:

TABLE	FIELD PATH	TYPE	FILL	SAMPLE
CommonSecurityLog	SourceIP	string	0.98	203.0.113.17
AppAudit_CL	Properties.client.addr	dynamic	0.71	198.51.100.4
Syslog	sshd: from <*> port	template	0.64	192.0.2.9

Classify by value, not only by name

Catches badly named columns such as cs4 or field7 that hold IP addresses.

IPv4 / IPv6 · UPN or email · SHA256 / MD5 · URL · Hostname / FQDN

Synonym dictionary

Column-name patterns mapped to ASIM field names. One file.

Template registry

Syslog message templates with each slot mapped to an ASIM field.

Entity index: one narrow table with ASIM column names

Re-run monthly, review the diff

`SourceTable` `SrcIpAddr` `DstIpAddr` `ActorUsername` `DvcHostname` `Url` `FileHash` `Domain`

Sources: Microsoft Learn, data lake KQL and notebooks; LevelBlue design

Checked 16 October 2026

© 2026 LevelBlue Intellectual Property. LevelBlue logo, and registered trademarks and service marks are the property of LevelBlue Intellectual Property and/or LevelBlue affiliated companies. All other marks are the property of their respective owners.

How far common fields can be trusted

Indicators hold up almost everywhere. Roles and outcomes need a parser or template.

	ASIM-parsed or Microsoft	CEF standard columns	CEF cs1 to cs6	Custom JSON _CL	Syslog, mapped templates	Syslog, long tail
IP address	High	High	Medium	High	High	Medium
File hash	High	High	Medium	High	High	Medium
URL or domain	High	High	Medium	Medium	Medium	Low
Hostname	High	Medium	Low	Medium	Medium	Low
Username	High	Medium	Low	Medium	Medium	Low
Role: source or target	High	Medium	Low	Low	Medium	Low
Event type and result	High	Low	Low	Low	Medium	Low

Expected confidence, based on how each field is found. Measure it per source with the coverage report before quoting it to anyone.

NORMALIZE VALUES, NOT ONLY FIELD NAMES

`CONTOSO\jsmith`

`jsmith@contoso.com` → `jsmith`

`S-1-5-21-…-1104`

`WS-0412.corp.contoso.com` → `ws-0412`

`ws-0412`

Lowercase · Split domain and user · Resolve SIDs · Keep FQDN and short name

WHAT YOU CAN SAY WITH CONFIDENCE

- ✓ This IP, hash or domain appeared, or did not appear, in these sources.
- ⊗ This user logged on and it succeeded, outside parsed sources.
- ⊗ Lateral movement across every source.

Sources: LevelBlue analysis of field extraction methods

Checked 16 October 2026

© 2026 LevelBlue Intellectual Property. LevelBlue logo, and registered trademarks and service marks are the property of LevelBlue Intellectual Property and/or LevelBlue affiliated companies. All other marks are the property of their respective owners.

Measure coverage for each source

Tier every source by how deep a query can see, then watch for formats that change underneath you.

TIER 1
ASIM-parsed

Full schema: roles, event type and result.

SrcIpAddr vs DstIpAddr | ActorUsername vs TargetUsername | EventType | EventResult

TIER 2
Entity only

Entities without roles or outcome.

IP | User | Host | Hash | URL

TIER 3
Search only

A literal string match, nothing more.

`where * has`

CHECKS ON EVERY RUN

Extraction rate
Share of rows per table with at least one entity

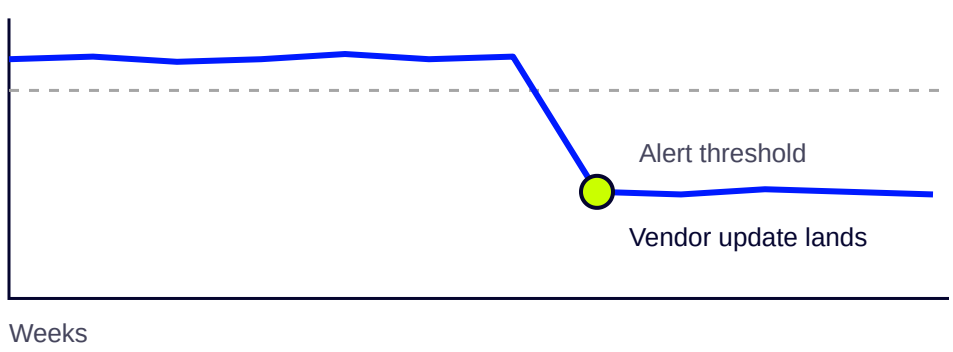
Fill rate per entity
How often each ASIM column is populated

Unmapped columns
High-fill columns the dictionary does not cover

Template match rate
Share of syslog lines matching a known template

Hand-checked sample
50 to 100 rows per source, reviewed by an analyst

Canary events
Known logon, IP and hash generated on purpose



Drift alert

A parser or mapping that stops matching returns fewer rows, not an error. A drop in extraction rate is usually the first sign that a vendor changed a format.

Chart is illustrative.

Sources: LevelBlue design for coverage reporting

Checked 16 October 2026

© 2026 LevelBlue Intellectual Property. LevelBlue logo, and registered trademarks and service marks are the property of LevelBlue Intellectual Property and/or LevelBlue affiliated companies. All other marks are the property of their respective owners.

What ASIM support in the data lake would change

Native support would replace custom work that every team hunting across third-party sources repeats today.

REQUESTED FROM THE SENTINEL PRODUCT GROUP

1. **Run ASIM parsers in lake KQL and KQL jobs**
   Unifying and source-specific parsers, or a lake-specific build of them.
2. **Document query-scoped functions in lake KQL**
   State whether let-defined functions are supported, so inlined parsers can stay readable.
3. **Lake tier for every native ASim table**
   So ingest-time normalization can land straight in the lake.
4. **ASIM output from filter-and-split transforms**
   Map fields to ASIM columns while routing data to the lake tier.
5. **A recommended normalization pattern for the lake**
   Published guidance for hunting across hundreds of third-party sources.

CUSTOM WORK TODAY	WITH THE REQUESTS	REQUEST
Copy ASIM parser logic into every lake query and KQL job	Removed	1, 2
Sync parser YAML from the Azure-Sentinel GitHub repo	Removed	1
Per-source DCRs for the highest-volume sources	Reduced	3, 4
Entity index for sources with no parser	Still needed	
Syslog template mining	Still needed	
Coverage and drift checks per source	Still needed	

Status on 16 October 2026: Sentinel's What's new page, from July 2025 to September 2026, lists no item for ASIM in the data lake or for function support in lake KQL. ASIM parser work continues on the analytics tier.

Sources: Microsoft Learn, What's new in Microsoft Sentinel; data lake KQL documentation

Checked 16 October 2026

© 2026 LevelBlue Intellectual Property. LevelBlue logo, and registered trademarks and service marks are the property of LevelBlue Intellectual Property and/or LevelBlue affiliated companies. All other marks are the property of their respective owners.

Worked example: Linux syslog into the entity index

No parsing at the collector. Every step below is plain KQL that runs in the lake.

RAW LINES AS THEY LAND: PROCESSNAME AND SYSLOGMESSAGE ONLY

Collector adds no parsing

```
web-07 sshd[2214]: Failed password for invalid user admin from 203.0.113.45 port 52114 ssh2
web-07 sshd[2214]: Accepted publickey for deploy from 198.51.100.20 port 40022 ssh2
web-07 sudo: deploy : TTY=pts/0 ; PWD=/home/deploy ; USER=root ; COMMAND=/usr/bin/systemctl restart nginx
```

> `TargetUsername = admin` `SrcIpAddr = 203.0.113.45` `EventType = Logon` `EventResult = Failure` `DvcHostname = web-07`

1 Find the templates

```
Syslog
| where TimeGenerated > ago(7d)
| where ProcessName == "sshd"
| reduce by SyslogMessage
    with threshold = 0.1
| top 20 by Count
```

Run per program. Test reduce on your lake; Drain3 in a notebook also works.

2 Record each template once

```
- process: sshd
  match: "Failed password for"
  fields:
    TargetUsername: 'for (?:invalid user )?(\S+) from'
    SrcIpAddr: 'from (\S+) port'
  EventType: Logon
  EventResult: Failure
```

One registry entry per template, kept in Git with the generator.

3 Generator writes plain KQL for each template

```
Syslog
| where TimeGenerated > ago(1h)
| where ProcessName == "sshd" and SyslogMessage has "Failed password for"
| extend TargetUsername = extract(@"for (?:invalid user )?(\S+) from", 1, SyslogMessage),
         SrcIpAddr = extract(@"from (\S+) port", 1, SyslogMessage)
| project TimeGenerated, SourceTable = "Syslog", EventVendor = "OpenSSH",
          EventType = "Logon", EventResult = "Failure", SrcIpAddr,
          TargetUsername, DvcHostname = Computer
```

One block per template, unioned into one KQL job. No functions, so it runs in lake KQL. Output columns use ASIM names.

4 Catch every unmatched line

```
Syslog
| where TimeGenerated > ago(1h)
| where not(SyslogMessage has_any
    ("Failed password for", "Accepted"))
| extend Ip = extract_all(
    @"\b(\d{1,3}(?:\.\d{1,3}){3})\b", SyslogMessage)
| mv-expand SrcIpAddr = Ip to typeof(string)
```

Unmatched lines still give up their IPs, as Tier 2 rows.

5 Hunt the index, not the raw table

```
HuntIndex_KQL_CL
| where SrcIpAddr == "203.0.113.45"
| summarize Events = count(),
    Sources = make_set(SourceTable)
    by EventType, EventResult
```

The KQL job writes HuntIndex_KQL_CL on a schedule. Analysts and the data lake MCP query this one table.

Sources: Microsoft Learn, KQL reduce, extract, extract_all and data lake KQL jobs; LevelBlue design

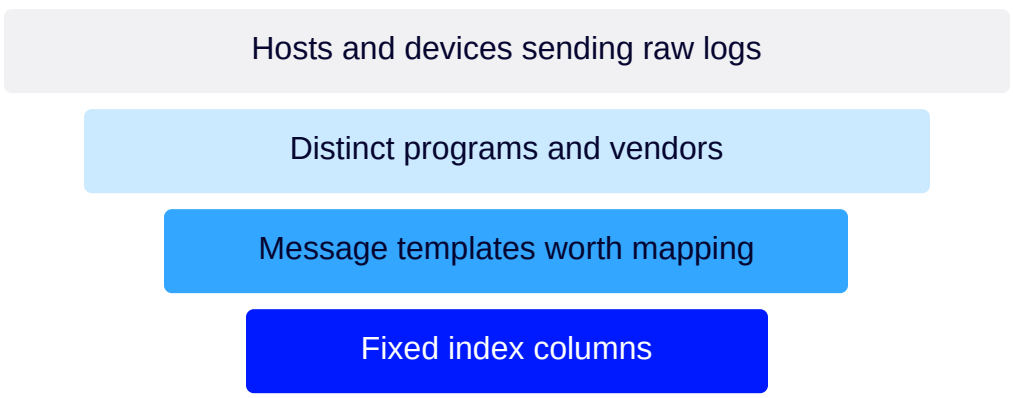
Checked 16 October 2026

© 2026 LevelBlue Intellectual Property. LevelBlue logo, and registered trademarks and service marks are the property of LevelBlue Intellectual Property and/or LevelBlue affiliated companies. All other marks are the property of their respective owners.

Scaling to hundreds of raw log sources

Maintain templates and a generator, not queries. Effort grows with formats, not hosts.

EFFORT FOLLOWS FORMATS, NOT HOSTS



Widths show direction, not measured counts.

GROUP SOURCES BY GRAMMAR

`Syslog`	ProcessName
`CommonSecurityLog`	DeviceVendor + DeviceProduct
`Custom _CL with RawData`	Source tag or first token of the line
`Custom _CL with JSON`	Schema fingerprint: sorted list of keys

500 web servers running the same sshd share one set of templates.

SHRINK THE UNMATCHED SHARE EVERY WEEK

Unmatched lines
Rows that only reached value regex

>

Mine templates
Notebook runs Drain3 per grammar group

>

Rank by volume
Largest new templates first

>

Analyst maps fields
Approves the template and its ASIM fields

>

Generator rebuilds jobs
KQL jobs regenerated from Git

Repeat: each pass moves the largest remaining formats from Tier 2 to Tier 1

WHERE RAW LINES LAND, PASS BY PASS



Illustrative shares, not measured.

WHAT RUNS WHERE

Template mining	Spark notebook	Weekly
Index build	One KQL job per grammar group	Hourly or daily
History backfill	Spark notebook	On demand
Coverage report	KQL job or notebook	Each build
Hunting	Entity index, interactive or MCP	Any time

RULES THAT KEEP IT FAST

- Filter on the grouping key and a `has` term before any regex.
- One job per grammar group, never one per host.
- Unmatched lines always go to value regex, so nothing is dropped.
- Stagger job schedules to stay inside the tenant job quota.
- Rebuild jobs from the registry; never hand-edit them.

Sources: Microsoft Learn, data lake KQL jobs and notebooks; LevelBlue design

Checked 16 October 2026

© 2026 LevelBlue Intellectual Property. LevelBlue logo, and registered trademarks and service marks are the property of LevelBlue Intellectual Property and/or LevelBlue affiliated companies. All other marks are the property of their respective owners.